

Video Game Programming: Game Physics

L06

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Physics engines

- In addition to a 3D rendering engine, 3D game engines usually integrate one or more physics engines to simulate physics rules
- For example, Unity allows to use 4 different physics engines:
 - For object-oriented projects
 - 2D -> Box2D engine integration
 - 3D -> Nvidia PhysX engine integration
 - For data-oriented projects
 - Base functions -> Unity Physics package
 - Additional functions -> Havok Physics for Unity package
- We will focus on 3D object-oriented projects

Nvidia PhysX engine

- Open-source physics engine
- Originally developed by Ageia, now owned by Nvidia
- Works on Windows, macOS, Linux, Android, iOS, and most of consoles
- Integrated by several game engines such as Unity, Unreal, Panda3D, Torque,...
- The same game can look and feel different with and without enabling PhysX (in fig.)



Rigid bodies

- In physics, a rigid body is a solid body in which deformation is zero or so small it can be neglected
- In physics engines for game engines, rigid bodies enable application of physics rules to game objects
- In Unity, Rigidbody component makes a game object a rigid body
- Rigid bodies are subject to gravity and are aware of collisions
- Game objects without a rigid body (and a collider) are like holograms
- Rigid bodies are typically moved by applying forces instead of directly translating them (e.g., changing position)
- Kinematic options to change position ignoring forces are available

Newton's laws of motion

- Rigid bodies are subject to Newton's laws of classical mechanics:
 1. Objects at rest will stay at rest and objects in motion will stay in motion unless external forces are applied
 - However, in a non-vacuum space, friction of materials should be considered
 - For optimization purposes, physics engines can consider very slow rigid bodies as sleeping and do not process them until a new force is applied
 2. $F(\text{orce}) = m(\text{ass}) * a(\text{cceleration})$, so $a = F/m$
 - Since $a = v(\text{elocity})/t(\text{ime})$ and $s(\text{pace}) = v * t$, $s = F/m * t^2$
 - As above, friction of materials should be considered
 3. Forces between two objects exist in equal magnitude and opposite direction
 - This influences behavior on collision

Colliders

- To detect collisions, game objects must have a collider
- In Unity, different types of colliders are available (Box, Capsule, Sphere) and they can be combined
- Mesh colliders can be used to increase precision at a performance cost
- Colliders can also be applied to game objects without a rigid body if the objects can collide with other objects but do not move
- These are considered static (instead of dynamic) colliders
- Some non-solid objects can have colliders to detect other colliders passing through without colliding but anyway triggering an event
- Script callbacks allow to write custom code for trigger or collision events

- Activation of trigger and collision events depends on rigid body and collider presence and settings
- In Unity, it also depends on game objects layers

- Activation of trigger and collision events depends on rigid body and collider presence and settings
- In Unity, it also depends on game objects layers

Trigger messages are sent upon collision						
	Static Collider	Rigidbody Collider	Kinematic Rigidbody Collider	Static Trigger Collider	Rigidbody Trigger Collider	Kinematic Rigidbody Trigger Collider
Static Collider					Y	Y
Rigidbody Collider				Y	Y	Y
Kinematic Rigidbody Collider				Y	Y	Y
Static Trigger Collider		Y	Y		Y	Y
Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y
Kinematic Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y

[illegible]

Collision detection methods

- Collision detection can be set to discrete or continuous
- Continuous collision detection (CCD) can be dynamic or speculative
- Dynamic CCD uses sweep-based / time of impact algorithm
- Speculative CCD tries to predict possible collision by extending AABB
- Performance cost: discrete < speculative CCD < dynamic CCD
- Speculative better for angular motion, but with ghost collision issues
- For more details and examples:
<https://docs.unity3d.com/Manual/ContinuousCollisionDetection.html>

Joints

- A joint connects a rigid body to another or to a fixed point in space
- Joints apply forces that move rigid bodies, and joint limits restrict that movement
- Different types of joints allow setting limits on different axes or on translation / rotation only
- Joints can be broken when forces above specified thresholds are applied
- In industrial, non-gaming, contexts, more complex physics articulations can be used instead of simpler joints

Character controllers

- Characters controlled by players should
 - respond to gravity (e.g., stay on the floor, unless player is jumping)
 - collide with other game objects (e.g., do not pass through wall)
 - influence other objects (e.g., push other objects away)
- However, since their movement should be in players' control
 - they may not rotate on some axes
 - they may not be influenced by collisions with other objects
- In Unity, a specific `CharacterController` component with these features is available
- `CharacterController` has also a virtual soft skin to avoid rigid collisions

Soft bodies

- Soft bodies are deformable objects, where relative distances between mesh points can change
- Typical examples are tissues and skinned characters
- Unity has a Cloth component to set constraints to a skinned mesh to simulate tissues
- It also has a wizard to setup skinned characters and ragdolls (in fig.)



Raycasting

- Raycasting consists in casting an invisible ray from an origin toward a direction
- This technique allows to check if the ray hits a collider
- Traditionally used for shooting games, it now has a much broader usage, e.g., blocking an NPC before it hits another one
- Several raycast options are available, as well as casting of 3D surfaces instead of a linear ray
- In Unity, raycasting is called programmatically using static methods of Physics class

Bibliografy

- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Murray, C# Game Programming Cookbook for Unity 3D, 2nd ed., CRC Press, 2021

